# SpacemiT K1 Asymmetry

- From BPI docs (Misspellings included):
  - "AI customized instruciton explored and designed in Cluster0"
  - "Cluster0 has 512KB TCM (Tight-Coupled Memory) for AI extention"
    - Avoids cache misses for tensor data during inference
    - Critical for bandwidth-hungry matrix operations
- Cluster 0 has dedicated AI hardware
- Several custom instructions specific to spacemiT compiler
  - Matrix Initialization
  - Convolutions
  - Holding Tensor Data
- spacemiT toolchain is pre-installed on BainbuOS (support out of the box)

# Cluster 0: Integrated Matrix Extension (IME)

- Matrix/tensor operations as native ISA instructions
- IME operates on top of the existing RVV
- Unconfirmed (reddit...) – Adds INT8, FP16, BF16 support, all standard ML inference data types
- Targets accelerated matrix multiply (the core operation in neural nets)
- Standardization in progress via RISC-V IME working group (link)
- Only supported by Cluster 0. Why?
  - Asymmetry – allows specialized hardware without paying the cost on every core
  - PPA + $ Reasons
    - Matrix register file is significant silicon area — cheaper to build once, instead of for both
    - Cuts area/power on the other 4 cores
    - TCM also expensive!

# SpacemiT K1 Asymmetry Example 1

- Using Bianbu OS here
- Running Vector Multiply-Add Dot Product demo program
  - From spacemiT github

```
shrek@spacemit-k1-x-deb1-board: ~/Documents/HPSC/riscv-ime-extension-spec-master/example
shrek@spacemit-k1-x-deb1-board:~/Documents/HPSC/riscv-ime-extension-spec-master/example$ gcc vmadot-gemm-demo.c -o run
vmadot-gemm-demo.c: Assembler messages:
vmadot-gemm-demo.c:47: Error: unrecognized opcode `vsetvli t0,zero,e8,mf8', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:51: Error: unrecognized opcode `vle8.v v0,(a3)', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:53: Error: unrecognized opcode `vsse8.v v0,(a4),t6', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:73: Error: unrecognized opcode `vsetvli t0,zero,e32,m2', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:74: Error: unrecognized opcode `vxor.vv v28,v28,v28', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:75: Error: unrecognized opcode `vsetvli t0,zero,e8,m1', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:77: Error: unrecognized opcode `vle8.v v0,(a2)', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:79: Error: unrecognized opcode `vle8.v v1,(a3)', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:81: Error: unrecognized opcode `vmadot v28,v0,v1', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:82: Error: unrecognized opcode `vsetvli t0,zero,e32,m2', extension `v' or `zve64x' or `zve32x' required
vmadot-gemm-demo.c:83: Error: unrecognized opcode `vse32.v v28,(a4)', extension `v' or `zve64x' or `zve32x' required
shrek@spacemit-k1-x-deb1-board:~/Documents/HPSC/riscv-ime-extension-spec-master/example$
```

# SpacemiT K1 Asymmetry Example 2

- Clear example of core asymmetry
- Program runs on cores 0-3, fails due to Illegal Instruction on cores 4-7

```
shrek@spacemit-k1-x-deb1-board: ~/Documents/HPSC/riscv-ime-extension-spec-master/example
shrek@spacemit-k1-x-deb1-board:~/Documents/HPSC/riscv-ime-extension-spec-master/example$ taskset -c 4 ./run
Illegal instruction (core dumped)
shrek@spacemit-k1-x-deb1-board:~/Documents/HPSC/riscv-ime-extension-spec-master/example$ taskset -c 5 ./run
Illegal instruction (core dumped)
shrek@spacemit-k1-x-deb1-board:~/Documents/HPSC/riscv-ime-extension-spec-master/example$ taskset -c 3 ./run
Test successful. CRef equal to C.
**********************************
matrix A:
   -8   -11    15   -60     6     6  -109    65
   94    62   -41     8    50   -54    10  -100
  -18   -99    -9    36    81   -97  -102    97
   15   107    -5    38    94   -81   -69   -22
matrix B:
  -93  -101   101    38
  -17   -94   112   124
   46   115    91    64
  -21  -126   -53  -126
   17   126   -30   125
  -86   101    96   120
   31   102   -13   -83
  112   -24   -52    42
matrix packB:
  -93   -17    46   -21    17   -86    31   112
 -101   -94   115  -126   126   101   102   -24
  101   112    91   -53   -30    96   -13   -52
   38   124    64  -126   125   120   -83    42
matrix CRef:
   6368    -189     938   20099
 -17246  -16779   10669    2368
  19608   -6770  -31093   -7047
   -281  -19783    2475   15563
matrix C:
   6368    -189     938   20099
 -17246  -16779   10669    2368
  19608   -6770  -31093   -7047
   -281  -19783    2475   15563
**********************************
shrek@spacemit-k1-x-deb1-board:~/Documents/HPSC/riscv-ime-extension-spec-master/example$
```

# Fault Injection Framework

- Google Earth image dataset
- Analyzed mappings over time
  - Injection can be performed early
- Proposed fault injection workflow:
  1. Begin and pause compression (get PID)
  2. Read virtual memory mappings
  3. Find physical frames of targeted regions
     - Start with heap/stack, find targeted structs in future
  4. Blanket region with SEU's (and/or targeted addresses) with /dev/mem
  5. Compression resumes: analyze failure
     - For shared libraries, fix faults manually (for now)
     - Continue injection during rest of runtime?

# Current Results

FaultInjection > mapping_examples > results4 > snapshots > maps_000001.txt

```
2ae14f2000-2ae14fe000 r-xp 00000000 b3:06 17                     /usr/bin/cjpeg
2ae14fe000-2ae14ff000 r--p 0000b000 b3:06 17                     /usr/bin/cjpeg
2ae14ff000-2ae1500000 rw-p 0000c000 b3:06 17                     /usr/bin/cjpeg
2ae1500000-2ae1521000 rw-p 00000000 00:00 0                      [heap]
3f8e62b000-3f8e6f0000 rw-p 00000000 00:00 0
3f8e6f0000-3f8e863000 r-xp 00000000 b3:06 29648                  /usr/lib/riscv64-linux-gnu/libc.so.6
3f8e863000-3f8e866000 r--p 00173000 b3:06 29648                  /usr/lib/riscv64-linux-gnu/libc.so.6
3f8e866000-3f8e868000 rw-p 00176000 b3:06 29648                  /usr/lib/riscv64-linux-gnu/libc.so.6
3f8e868000-3f8e874000 rw-p 00000000 00:00 0
3f8e874000-3f8e8ea000 r-xp 00000000 b3:06 30500                  /usr/lib/riscv64-linux-gnu/libjpeg.so.8.2.2
3f8e8ea000-3f8e8eb000 r--p 00076000 b3:06 30500                  /usr/lib/riscv64-linux-gnu/libjpeg.so.8.2.2
3f8e8eb000-3f8e8ec000 rw-p 00077000 b3:06 30500                  /usr/lib/riscv64-linux-gnu/libjpeg.so.8.2.2
3f8e8ec000-3f8e8ee000 rw-p 00000000 00:00 0
3f8e903000-3f8e905000 r--p 00000000 00:00 0                      [vvar]
3f8e905000-3f8e907000 r-xp 00000000 00:00 0                      [vdso]
3f8e907000-3f8e92a000 r-xp 00000000 b3:06 28696                  /usr/lib/riscv64-linux-gnu/ld-linux-riscv64-lp64d.so.1
3f8e92a000-3f8e92c000 r--p 00022000 b3:06 28696                  /usr/lib/riscv64-linux-gnu/ld-linux-riscv64-lp64d.so.1
3f8e92c000-3f8e92d000 rw-p 00024000 b3:06 28696                  /usr/lib/riscv64-linux-gnu/ld-linux-riscv64-lp64d.so.1
3f8e92d000-3f8e92e000 rw-p 00000000 00:00 0
3ffa369000-3ffa38b000 rw-p 00000000 00:00 0                      [stack]
```

# Current Results

```
shrek@spacemit-k1-x-deb1-board:~/Documents/HPSC/HPSC-Fault-Tolerance$ sudo ./FaultInjection/inject_faults.sh Dat
aset/05_large.ppm test.jpg 75 stack --DRY_RUN 0 --FLIPS_PER_PFN 1
Using QUALITY=75 TARGET_MAPPING=stack DRY_RUN=0 FLIPS_PER_PFN=1 PFN_LIST=/home/shrek/Documents/HPSC/HPSC-Fault-T
olerance/FaultInjection/pfns.txt
Started cjpeg with PID 8032
Paused PID 8032
Considering mapping: 3ff9707000-3ff9728000 rw-p 00000000 00:00 0                          [stack]
Found PFN=0x230fd8 for vaddr=0x3ff9725000
Found PFN=0x23d9f4 for vaddr=0x3ff9726000
Found PFN=0x2313c9 for vaddr=0x3ff9727000
Scanned 33 pages across 1 matching mappings. Present=3 Swapped=0 PFN_zero=0 PFNs_written=3 -> /home/shrek/Docume
nts/HPSC/HPSC-Fault-Tolerance/FaultInjection/pfns.txt.tmp
Wrote deduplicated PFN list to /home/shrek/Documents/HPSC/HPSC-Fault-Tolerance/FaultInjection/pfns.txt
pread failed at 0x230fd8984: Operation not permitted
pread failed at 0x2313c906f: Operation not permitted
pread failed at 0x23d9f42a2: Operation not permitted
Processed 3 PFNs and attempted 0 flips (live mode).
Resumed PID 8032
cjpeg exit code: 0
```